Homework 4:
NLTK: Corpus Linguistics

Robert Litschko*
Symbolische Programmiersprache

Due: Wednesday November 12, 2025, 12:00 (noon)

In this exercise you will:

- practice object oriented programming
- use NLTK to analyze text and perform basic corpus linguistics
- implement a language guesser

Exercise 1: Object-oriented programming II [3 points]

For this exercise we will use the solution of last weeks exercise as a starting point. Please implement your solution in the `hw04_nltk/document.py` file.

This part of the homework will be graded using unit tests by running:

```
python3 -m unittest -v src.hw04_nltk.test_document
```

Implement the following methods:

1. Inheritance [1 points]: Modify the class `PDFDocument` to make it inherit methods and attributes from `TextDocument`.
2. Override the constructor in `PDFDocument` [1 points]: It should accept a `docid` and a `filepath` variable (string) that points to the location of a pdf file on disk[1]. You should first use the `load_pdf()` function provided by us to extract the content of the pdf file[2] and then pass the text and the document id to the parent constructor.

*Credit: Exercises are based on previous iterations from Katerina Kalouli.
[1]For example: `/home/usr/myfile.pdf`
[2]For this to work you need to install PyPDF with `'pip install PyPDF2'`, refer to last weeks exercise for more information on installing python packages.

3. Aggregation [1 points]: Create a class `Author` with the attributes `firstname`, `lastname` and `age`.[3] Extend the constructor of `PDFDocument` to by adding an additional parameter and instance attribute `author`.

Exercise 2: Lexical information [8 points]

Take a look at `hw04_nltk/analyze.py`. In this exercise you will have to implement some methods in class Analyzer, that can analyze any text. In this exercise we will use `analyze.py` to compute basic statistics on `ada_lovelace.txt`.

This part of the homework will be graded using unit tests by running:

```
python3 -m unittest -v src.hw04_nltk.test_analyzer
```

Implement the following methods:

1. __init__(self, path) - should read the file text and store the text as an attribute, create the list of of words (use `nltk.word_tokenize` to tokenize the text), and calculate frequency distribution of words (use `nltk.FreqDist`)
2. numberOfTokens(self) [1 points] – should return the number of tokens in the text
3. numberOfSents(self) [1 points] - should return the number of sentences in the text (use `nltk.sent_tokenize`)
4. probDist(self) [1 point] - returns a dictionary of probability distribution for each word. (Note: probability of a word = frequency of the word / total number of tokens)
5. topFiveFreqTokens(self) [1 point] - returns a list of strings of the top five most frequent tokens in the text (sorted by frequency from high to low and alphabetically in case of ties)
6. lexicalDiversity(self) [1 points] – returns the lexical diversity of the text.
7. getYears(self) [1 points] - returns a list of distinct "words" that are in fact four-digit numbers.
8. numberOfHapaxes(self) [1 points] – returns the number of hapaxes in the text
9. avWordLength(self) [1 points] – returns the average word length in the text, rounded to two decimal places.

[3] You only need to implement the constructor.

Exercise 3: Language Guesser [4 points]

Implement a language guesser that determines the language it thinks the text is written in. The decision should be based on the frequency of bigrams in each language. Take a look at the file `hw04_nltk/model_lang.py`. In this exercise you will have to complete some methods to make it work (you can also take a look at the last slides of our Tuesday session to get some more help).

This homework will be graded using unit tests by running:

```
python3 -m unittest -v src.hw04_nltk.test_lang_guesser.py
```

Complete the following tasks:

1. Complete the class method `build_language_models(self)`. This method should return a conditional frequency distribution where [2 points]:
   a) the languages are the conditions
   b) the values are the frequency distribution of lower case bigrams in corresponding language
2. Complete the class method `guess_language(self,language_model_cfd, text)` [2 points]:
   a) it should calculate the overall score of a given text based on the frequency of bigrams accessible by `language_model_cfd[language].freq(bigram)` .
   b) it should return a tuple `(most_likely_language, confidence_score)` for a given text according to the scores, where `confidence_score`, rounded to two decimal places.